

13.2.2 Bullet

Bullet is an open source collision detection and physics library used by both the game and film industries. Its collision engine is integrated with its dynamics simulation, but hooks are provided so that the collision system can be used stand-alone or integrated with other physics engines. It supports *continuous collision detection* (CCD)—also known as *time of impact* (TOI) collision detection—which, as we’ll see below, can be extremely helpful when a simulation includes small, fast-moving objects. The Bullet SDK is available for download at <http://code.google.com/p/bullet/>, and the Bullet wiki is located at http://www.bulletphysics.com/mediawiki-1.5.8/index.php?title=Main_Page.

13.2.3 TrueAxis

TrueAxis is another collision/physics SDK. It is free for non-commercial use. You can learn more about TrueAxis at <http://trueaxis.com>.

13.2.4 PhysX

PhysX started out as a library called Novodex, produced and distributed by Ageia as part of their strategy to market their dedicated physics coprocessor. It was bought by NVIDIA and retooled so that it can run using NVIDIA’s GPUs as a coprocessor. (It can also run entirely on a CPU, without GPU support.) It is available at http://www.nvidia.com/object/nvidia_physx.html. Part of Ageia’s and NVIDIA’s marketing strategy has been to provide the CPU version of the SDK entirely for free, in order to drive the physics coprocessor market forward. Developers can also pay a fee to obtain full source code and the ability to customize the library as needed. PhysX is now combined with APEX, NVIDIA’s scalable multiplatform dynamics framework. PhysX/APEX is available for Windows, Linux, Mac, Android, Xbox 360, PlayStation 3, Xbox One, PlayStation 4 and Wii.

13.2.5 Havok

Havok is the gold standard in commercial physics SDKs, providing one of the richest feature sets available and boasting excellent performance characteristics on all supported platforms. (It’s also the most expensive solution.) Havok is comprised of a core collision/physics engine, plus a number of optional add-on products including a vehicle physics system, a system for modeling destructible environments and a fully featured animation SDK with direct integration into Havok’s rag doll physics system. It is available on Xbox 360,

PlayStation 3, Xbox One, PlayStation 4, PlayStation Vita, Wii, Wii U, Windows 8, Android, Apple Mac and iOS. You can learn more about Havok at <http://www.havok.com>.

13.2.6 Physics Abstraction Layer (PAL)

The Physics Abstraction Layer (PAL) is an open source library that allows developers to work with more than one physics SDK on a single project. It provides hooks for PhysX (Novodex), Newton, ODE, OpenTissue, Tokamak, TrueAxis and a few other SDKs. You can read more about PAL at <http://www.adrianboeing.com/pal/index.html>.

13.2.7 Digital Molecular Matter (DMM)

Pixelux Entertainment S.A., located in Geneva, Switzerland, has produced a unique physics engine that uses finite element methods to simulate the dynamics of deformable bodies and breakable objects, called Digital Molecular Matter (DMM). The engine has both an offline and a runtime component. It was released in 2008 and can be seen in action in LucasArts' *Star Wars: The Force Unleashed*. A discussion of deformable body mechanics is beyond our scope here, but you can read more about DMM at <http://www.pixeluxentertainment.com>.

13.3 The Collision Detection System

The primary purpose of a game engine's collision detection system is to determine whether any of the objects in the game world have come into *contact*. To answer this question, each logical object is represented by one or more geometric *shapes*. These shapes are usually quite simple, such as spheres, boxes and capsules. However, more complex shapes can also be used. The collision system determines whether or not any of the shapes are *intersecting* (i.e., overlapping) at any given moment in time. So a collision detection system is essentially a glorified geometric intersection tester.

Of course, the collision system does more than answer yes/no questions about shape intersection. It also provides relevant information about the nature of each contact. Contact information can be used to prevent unrealistic visual anomalies on-screen, such as objects *interpenetrating* one another. This is generally accomplished by moving all interpenetrating objects apart prior to rendering the next frame. Collisions can provide *support* for an object—one or more contacts that together allow the object to come to rest, in equilibrium with gravity and/or any other forces acting on it. Collisions can also be used

for other purposes, such as to cause a missile to explode when it strikes its target or to give the player character a health boost when he passes through a floating health pack. A rigid body dynamics simulation is often the most demanding client of the collision system, using it to mimic physically realistic behaviors like bouncing, rolling, sliding and coming to rest. But, of course, even games that have no physics system can still make heavy use of a collision detection engine.

In this chapter, we'll go on a brief high-level tour of how collision detection engines work. For an in-depth treatment of this topic, a number of excellent books on real-time collision detection are available, including [14], [48] and [11].

13.3.1 Collidable Entities

If we want a particular logical object in our game to be capable of colliding with other objects, we need to provide it with a *collision representation*, describing the object's shape and its position and orientation in the game world. This is a distinct data structure, separate from the object's *gameplay representation* (the code and data that define its role and behavior in the game) and separate from its *visual representation* (which might be an instance of a triangle mesh, a subdivision surface, a particle effect or some other visual representation).

From the point of view of detecting intersections, we generally favor shapes that are geometrically and mathematically simple. For example, a rock might be modeled as a sphere for collision purposes; the hood of a car might be represented by a rectangular box; a human body might be approximated by a collection of interconnected *capsules* (pill-shaped volumes). Ideally, we should resort to a more complex shape only when a simpler representation proves inadequate to achieve the desired behavior in the game. Figure 13.1 shows a few examples of using simple shapes to approximate object volumes for collision detection purposes.

Havok uses the term *collidable* to describe a distinct, rigid object that can take part in collision detection. It represents each collidable with an instance of the C++ class `hkpCollidable`. PhysX calls its rigid objects *actors* and represents them as instances of the class `NxActor`. In both of these libraries, a collidable entity contains two basic pieces of information—a *shape* and a *transform*. The shape describes the collidable's geometric form, and the transform describes the shape's position and orientation in the game world. Collidables need transforms for three reasons:

1. Technically speaking, a shape only describes the form of an object (i.e., whether it is a sphere, a box, a capsule or some other kind of volume).

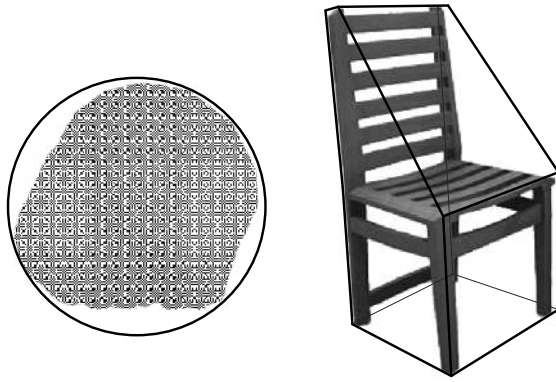


Figure 13.1. Simple geometric shapes are often used to approximate the collision volumes of the objects in a game.

It may also describe the object's size (e.g., the radius of a sphere or the dimensions of a box). But a shape is usually defined with its center at the origin and in some sort of canonical orientation relative to the coordinate axes. To be useful, a shape must therefore be transformed in order to position and orient it appropriately in world space.

2. Many of the objects in a game are dynamic. Moving an arbitrarily complex shape through space could be expensive if we had to move the *features* of the shape (vertices, planes, etc.) individually. But with a transform, any shape can be moved in space inexpensively, no matter how simple or complex the shape's features may be.
3. The information describing some of the more complex kinds of shapes can take up a nontrivial amount of memory. So, it can be beneficial to permit more than one collidable to share a single shape description. For example, in a racing game, the shape information for many of the cars might be identical. In that case, all of the car collidables in the game can share a single car shape.

Any particular object in the game may have no collidable at all (if it doesn't require collision detection services), a single collidable (if the object is a simple rigid body) or multiple collidables (each representing one rigid component of an articulated robot arm, for example).

13.3.2 The Collision/Physics World

A collision system typically keeps track of all of its collidable entities via a singleton data structure known as the *collision world*. The collision world is a complete representation of the game world designed explicitly for use by the collision detection system. Havok's collision world is an instance of the class `hkpWorld`. Likewise, the PhysX world is an instance of `NxScene`. ODE uses an instance of class `dSpace` to represent the collision world; it is actually the root of a hierarchy of geometric volumes representing all the collidable shapes in the game.

Maintaining all collision information in a private data structure has a number of advantages over attempting to store collision information with the game objects themselves. For one thing, the collision world need only contain collidables for those game objects that can potentially collide with one another. This eliminates the need for the collision system to iterate over any irrelevant data structures. This design also permits collision data to be organized in the most efficient manner possible. The collision system can take advantage of cache coherency to maximize performance, for example. The collision world is also an effective encapsulation mechanism, which is generally a plus from the perspectives of understandability, maintainability, testability and the potential for software reuse.

13.3.2.1 The Physics World

If a game has a rigid body dynamics system, it is usually tightly integrated with the collision system. It typically shares its “world” data structure with the collision system, and each rigid body in the simulation is usually associated with a single collidable in the collision system. This design is commonplace among physics engines because of the frequent and detailed collision queries required by the physics system. It's typical for the physics system to actually *drive* the operation of the collision system, instructing it to run collision tests at least once, and sometimes multiple times, per simulation time step. For this reason, the collision world is often called the *collision/physics world* or sometimes just the *physics world*.

Each dynamic rigid body in the physics simulation is usually associated with a single collidable object in the collision system (although not all collidables need be dynamic rigid bodies). For example, in Havok, a rigid body is represented by an instance of the class `hkpRigidBody`, and each rigid body has a pointer to exactly one `hkpCollidable`. In PhysX, the concepts of collidable and rigid body are comingled—the `NxActor` class serves both purposes (although the physical properties of the rigid body are stored separately, in

an instance of `NxBodyDesc`). In both SDKs, it is possible to tell a rigid body that its location and orientation are to be fixed in space, meaning that it will be omitted from the dynamics simulation and will serve as a collidable only.

Despite this tight integration, most physics SDKs do make at least some attempt to separate the collision library from the rigid body dynamics simulation. This permits the collision system to be used as a stand-alone library (which is important for games that don't need physics but do need to detect collisions). It also means that a game studio could *theoretically* replace a physics SDK's collision system entirely, without having to rewrite the dynamics simulation. (Practically speaking, this may be a bit harder than it sounds!)

13.3.3 Shape Concepts

A rich body of mathematical theory underlies the everyday concept of *shape* (see <http://en.wikipedia.org/wiki/Shape>). For our purposes, we can think of a shape simply as a region of space described by a boundary, with a definite *inside* and *outside*. In two dimensions, a shape has area, and its boundary is defined either by a curved line or by three or more straight edges (in which case it's a *polygon*). In three dimensions, a shape has volume, and its boundary is either a curved surface or is composed of polygons (in which case it is called a *polyhedron*).

It's important to note that some kinds of game objects, like terrain, rivers or thin walls, might be best represented by *surfaces*. In three-space, a surface is a two-dimensional geometric entity with a *front* and a *back* but no inside or outside. Examples include planes, triangles, subdivision surfaces and surfaces constructed from a group of connected triangles or other polygons. Most collision SDKs provide support for surface primitives and extend the term *shape* to encompass both closed volumes and open surfaces.

It's commonplace for collision libraries to allow surfaces to be given volume via an optional extrusion parameter. Such a parameter specifies how "thick" a surface should be. Doing this helps reduce the occurrence of missed collisions between small, fast-moving objects and infinitesimally thin surfaces (the so-called "bullet through paper" problem—see Section 13.3.5.7).

13.3.3.1 Intersection

We all have an intuitive notion of what an *intersection* is. Technically speaking, the term comes from set theory ([http://en.wikipedia.org/wiki/Intersection_\(set_theory\)](http://en.wikipedia.org/wiki/Intersection_(set_theory))). The intersection of two sets is comprised of the subset of members that are common to both sets. In geometrical terms, the intersection between two shapes is just the (infinitely large!) set of all points that lie inside both shapes.

13.3.3.2 Contact

In games, we're not usually interested in finding the intersection in the strictest sense, as a set of points. Instead, we want to know simply whether or not two objects are intersecting. In the event of a collision, the collision system will usually provide additional information about the nature of the contact. This information allows us to separate the objects in a physically plausible and efficient way, for example.

Collision systems usually package contact information into a convenient data structure that can be instanced for each contact detected. For example, Havok returns contacts as instances of the class `hkContactPoint`. Contact information often includes a *separating vector*—a vector along which we can slide the objects in order to efficiently move them out of collision. It also typically contains information about which two collidables were in contact, including which individual shapes were intersecting and possibly even which individual features of those shapes were in contact. The system may also return additional information, such as the velocity of the bodies projected onto the separating normal.

13.3.3.3 Convexity

One of the most important concepts in the field of collision detection is the distinction between *convex* and *non-convex* (i.e., *concave*) shapes. Technically, a convex shape is defined as one for which no ray originating inside the shape will pass through its surface more than once. A simple way to determine if a shape is convex is to imagine shrink-wrapping it with plastic film—if it's convex, no air pockets will be left under the film. So in two dimensions, circles, rectangles and triangles are all convex, but Pac Man is not. The concept extends equally well to three dimensions.

The property of *convexity* is important because, as we'll see, it's generally simpler and less computationally intensive to detect intersections between convex shapes than concave ones. See <http://en.wikipedia.org/wiki/Convex> for more information about convex shapes.

13.3.4 Collision Primitives

Collision detection systems can usually work with a relatively limited set of shape types. Some collision systems refer to these shapes as *collision primitives* because they are the fundamental building blocks out of which more complex shapes can be constructed. In this section, we'll take a brief look at some of the most common types of collision primitives.

13.3.4.1 Spheres

The simplest three-dimensional volume is a sphere. And as you might expect, spheres are the most efficient kind of collision primitive. A sphere is represented by a center point and a radius. This information can be conveniently packed into a four-element floating-point vector—a format that works particularly well with SIMD math libraries.

13.3.4.2 Capsules

A capsule is a pill-shaped volume, composed of a cylinder and two hemispherical end caps. It can be thought of as a *swept sphere*—the shape that is traced out as a sphere moves from point A to point B. (There are, however, some important differences between a static capsule and a sphere that sweeps out a capsule-shaped volume over time, so the two are not identical.) Capsules are often represented by two points and a radius (Figure 13.2). Capsules are more efficient to intersect than cylinders or boxes, so they are often used to model objects that are roughly cylindrical, such as the limbs of a human body.

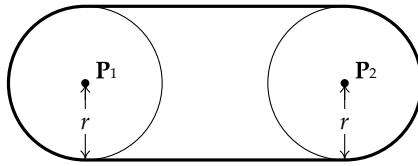


Figure 13.2. A capsule can be represented by two points and a radius.

13.3.4.3 Axis-Aligned Bounding Boxes

An axis-aligned bounding box (AABB) is a rectangular volume (technically known as a *cuboid*) whose faces are parallel to the axes of the coordinate system. Of course, a box that is axis-aligned in one coordinate system will not necessarily be axis-aligned in another. So we can only speak about an AABB in the context of the particular coordinate frame(s) with which it aligns.

An AABB can be conveniently defined by two points: one containing the minimum coordinates of the box along each of the three principal axes and the other containing its maximum coordinates. This is depicted in Figure 13.3.

The primary benefit of axis-aligned boxes is that they can be tested for interpenetration with other axis-aligned boxes in a highly efficient manner. The big limitation of using AABBs is that they must remain axis-aligned at all times if their computational advantages are to be maintained. This means that if an AABB is used to approximate the shape of an object in the game,

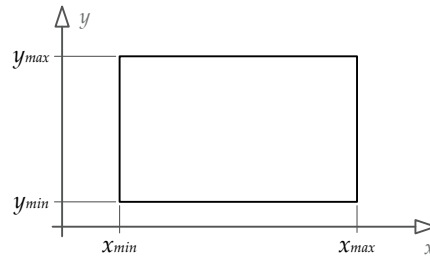


Figure 13.3. An axis-aligned box.

the AABB will have to be recalculated whenever that object rotates. Even if an object is roughly box-shaped, its AABB may degenerate into a very poor approximation to its shape when the object rotates off-axis. This is shown in Figure 13.4.

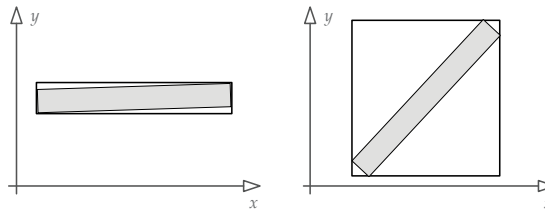


Figure 13.4. An AABB is only a good approximation to a box-shaped object when the object's principal axes are roughly aligned with the coordinated system's axes.

13.3.4.4 Oriented Bounding Boxes

If we permit an axis-aligned box to rotate relative to its coordinate system, we have what is known as an oriented bounding box (OBB). It is often represented by three half-dimensions (half-width, half-depth and half-height) and a transformation, which positions the center of the box and defines its orientation relative to the coordinate axes. Oriented boxes are a commonly used collision primitive because they do a better job at fitting arbitrarily oriented objects, yet their representation is still quite simple.

13.3.4.5 Discrete Oriented Polytopes (DOP)

A discrete oriented polytope (DOP) is a more-general case of the AABB and OBB. It is a convex polytope that approximates the shape of an object. A DOP can be constructed by taking a number of planes at infinity and sliding them along their normal vectors until they come into contact with the object whose

shape is to be approximated. An AABB is a 6-DOP in which the plane normals are taken parallel to the coordinate axes. An OBB is also a 6-DOP in which the plane normals are parallel to the object's natural principal axes. A k -DOP is constructed from an arbitrary number of planes k . A common method of constructing a DOP is to start with an OBB for the object in question and then bevel the edges and/or corners at 45 degrees with additional planes in an attempt to yield a tighter fit. An example of a k -DOP is shown in Figure 13.5.

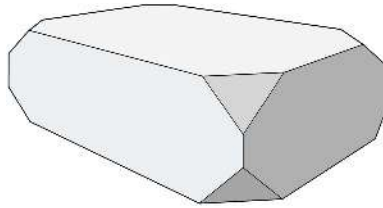


Figure 13.5. An OBB that has been beveled on all eight corners is known as a 14-DOP.

13.3.4.6 Arbitrary Convex Volumes

Most collision engines permit arbitrary convex volume to be constructed by a 3D artist in a package like Maya. The artist builds the shape out of polygons (triangles or quads). An offline tool analyzes the triangles to ensure that they actually do form a convex polyhedron. If the shape passes the convexity test, its triangles are converted into a collection of planes (essentially a k -DOP), represented by k plane equations, or k points and k normal vectors. (If it is found to be non-convex, it can still be represented by a polygon soup—described in the next section.) This approach is depicted in Figure 13.6.

Convex volumes are more expensive to intersection-test than the simpler geometric primitives we've discussed thus far. However, as we'll see in Section

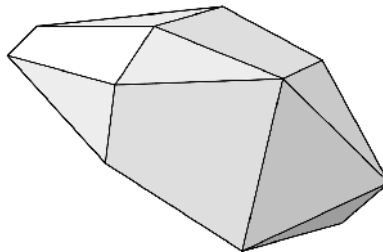


Figure 13.6. An arbitrary convex volume can be represented by a collection of intersecting planes.

13.3.5.5, certain highly efficient intersection-finding algorithms such as GJK are applicable to these shapes because they are convex.

13.3.4.7 Poly Soup

Some collision systems also support totally arbitrary, non-convex shapes. These are usually constructed out of triangles or other simple polygons. For this reason, this type of shape is often called a *polygon soup*, or *poly soup* for short. Poly soups are often used to model complex static geometry, such as terrain and buildings (Figure 13.7).

As you might imagine, detecting collisions with a poly soup is the most expensive kind of collision test. In effect, the collision engine must test every individual triangle, and it must also properly handle spurious intersections with triangle edges that are shared between adjacent triangles. As a result, most games try to limit the use of poly soup shapes to objects that will not take part in the dynamics simulation.

Does a Poly Soup Have an Inside?

Unlike convex and simple shapes, a poly soup does not necessarily represent a volume—it can represent an open surface as well. Poly soup shapes often don't include enough information to allow the collision system to differentiate between a closed volume and an open surface. This can make it difficult to know in which direction to push an object that is interpenetrating a poly soup in order to bring the two objects out of collision.

Thankfully, this is by no means an intractable problem. Each triangle in

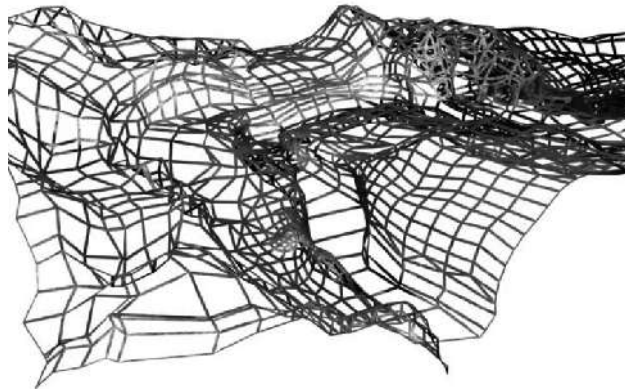


Figure 13.7. A poly soup is often used to model complex static surfaces such as terrain or buildings.

a poly soup has a front and a back, as defined by the winding order of its vertices. Therefore, it is possible to carefully construct a poly soup shape so that all of the polygons' vertex winding orders are consistent (i.e., adjacent triangles always "face" in the same direction). This gives the entire poly soup a notion of "front" and "back." If we also store information about whether a given poly soup shape is open or closed (presuming that this fact can be ascertained by offline tools), then for closed shapes, we can interpret "front" and "back" to mean "outside" and "inside" (or vice versa, depending on the conventions used when constructing the poly soup).

We can also "fake" an inside and outside for certain kinds of *open* poly soup shapes (i.e., surfaces). For example, if the terrain in our game is represented by an open poly soup, then we can decide arbitrarily that the front of the surface always points away from the Earth. This implies that "front" should always correspond to "outside." Practically speaking, to make this work, we would probably need to customize the collision engine in some way in order to make it aware of our particular choice of conventions.

13.3.4.8 Compound Shapes

Some objects that cannot be adequately approximated by a single shape can be approximated well by a *collection* of shapes. For example, a chair might be modeled out of two boxes—one for the back of the chair and one enclosing the seat and all four legs. This is shown in Figure 13.8.

A compound shape can often be a more-efficient alternative to a poly soup for modeling non-convex objects; two or more convex volumes can often outperform a single poly soup shape. What's more, some collision systems can take advantage of the convex bounding volume of the compound shape as a whole when testing for collisions. In Havok, this is called *midphase* collision detection. As the example in Figure 13.9 shows, the collision system first tests the convex bounding volumes of the two compound shapes. If they do not intersect, the system needn't test the subshapes for collisions at all.

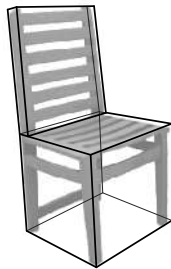


Figure 13.8. A chair can be modeled using a pair of interconnected box shapes.

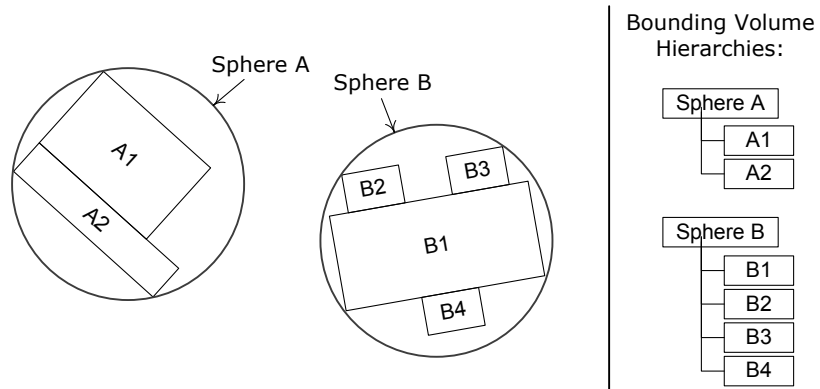


Figure 13.9. A collision system need only test the subshapes of a pair of compound shapes when their convex bounding volumes (in this case, Sphere A and Sphere B) are found to be intersecting.

13.3.5 Collision Testing and Analytical Geometry

A collision system makes use of *analytical geometry*—mathematical descriptions of three-dimensional volumes and surfaces—in order to detect intersections between shapes computationally. See http://en.wikipedia.org/wiki/Analytic_geometry for more details on this profound and broad area of research. In this section, we'll briefly introduce the concepts behind analytical geometry, show a few common examples and then discuss the generalized GJK intersection testing algorithm for arbitrary convex polyhedra.

13.3.5.1 Point versus Sphere

We can determine whether a point $\mathbf{p}$ lies within a sphere by simply forming the separation vector $\mathbf{s}$ between the point and the sphere's center $\mathbf{c}$ and then checking its length. If it is greater than the radius of the sphere r , then the point lies outside the sphere; otherwise, it lies inside:

$$\mathbf{s} = \mathbf{c} - \mathbf{p};$$

$$\text{if } |\mathbf{s}| \leq r, \text{ then } \mathbf{p} \text{ is inside.}$$

13.3.5.2 Sphere versus Sphere

Determining if two spheres intersect is almost as simple as testing a point against a sphere. Again, we form a vector $\mathbf{s}$ connecting the center points of the two spheres. We take its length and compare it with the sum of the radii of the two spheres. If the length of the separating vector is less than or equal to the sum of the radii, the spheres intersect; otherwise, they do not:

$$\mathbf{s} = \mathbf{c}_1 - \mathbf{c}_2; \tag{13.1}$$

$$\text{if } |\mathbf{s}| \leq (r_1 + r_2), \text{ then spheres intersect.}$$

To avoid the square root operation inherent in calculating the length of vector $\mathbf{s}$, we can simply square the entire equation. So Equation (13.1) becomes

$$\begin{aligned}\mathbf{s} &= \mathbf{c}_1 - \mathbf{c}_2; \\ |\mathbf{s}|^2 &= \mathbf{s} \cdot \mathbf{s}; \\ \text{if } |\mathbf{s}|^2 &\leq (r_1 + r_2)^2, \text{ then spheres intersect.}\end{aligned}$$

13.3.5.3 The Separating Axis Theorem

Most collision detection systems make heavy use of a theorem known as the *separating axis theorem* (http://en.wikipedia.org/wiki/Separating_axis_theorem). It states that if an axis can be found along which the *projections* of two convex shapes do not overlap, then we can be certain that the two shapes do not intersect at all. If such an axis does not exist *and* the shapes are convex, then we know for certain that they do intersect. (If the shapes are concave, then they may not be interpenetrating despite the lack of a separating axis. This is one reason why we tend to favor convex shapes in collision detection.)

This theorem is easiest to visualize in two dimensions. Intuitively, it says that if a line can be found, such that object A is entirely on one side of the line and object B is entirely on the other side, then objects A and B do not overlap. Such a line is called a *separating line*, and it is always *perpendicular* to the separating axis. So once we've found a separating line, it's a lot easier to convince ourselves that the theory is in fact correct by looking at the projections of our shapes onto the axis that is perpendicular to the separating line.

The projection of a two-dimensional *convex* shape onto an axis acts like the shadow that the object would leave on a thin wire. It is always a line segment, lying on the axis, that represents the maximum extents of the object in the direction of the axis. We can also think of a projection as a minimum and maximum coordinate along the axis, which we can write as the fully closed interval $[c_{\min}, c_{\max}]$. As you can see in Figure 13.10, when a separating line exists between two shapes, their projections do not overlap along the separating axis. However, the projections may overlap along other, non-separating axes.

In three dimensions, the separating line becomes a separating plane, but the separating axis is still an axis (i.e., an infinite line). Again, the projection of a three-dimensional *convex* shape onto an axis is a line segment, which we can represent by the fully closed interval $[c_{\min}, c_{\max}]$.

Some types of shapes have properties that make the potential separating axes obvious. To detect intersections between two such shapes A and B, we can project the shapes onto each potential separating axis in turn and then check whether or not the two projection intervals, $[c_{\min}^A, c_{\max}^A]$ and $[c_{\min}^B, c_{\max}^B]$,

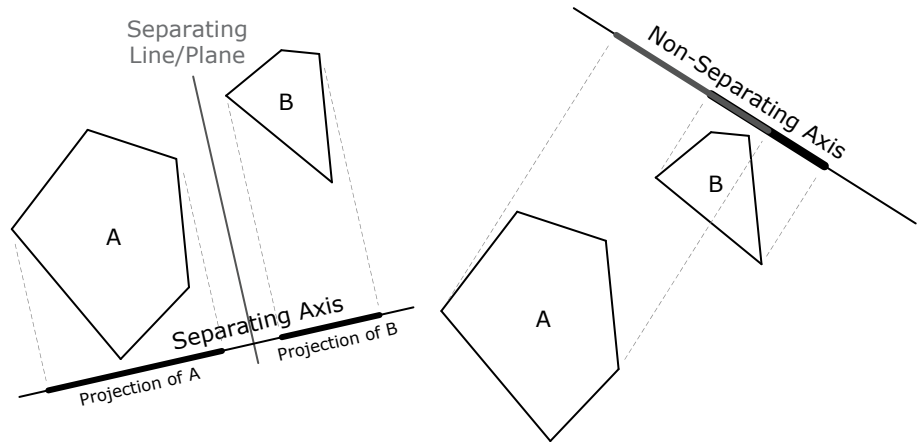


Figure 13.10. The projections of two shapes onto a separating axis are always two disjoint line segments. The projections of these same shapes onto a non-separating axis are not necessarily disjoint. If no separating axis exists, the shapes intersect.

are disjoint (i.e., do not overlap). In math terms, the intervals are disjoint if $c_{\max}^A < c_{\min}^B$ or if $c_{\max}^B < c_{\min}^A$. If the projection intervals along one of the potential separating axes are disjoint, then we've found a separating axis, and we know the two shapes do not intersect.

One example of this principle in action is the sphere-versus-sphere test. If two spheres do not intersect, then the axis parallel to the line segment joining the spheres' center points will always be a valid separating axis (although other separating axes may exist, depending on how far apart the two spheres are). To visualize this, consider the limit when the two spheres are just about to touch but have not yet come into contact. In that case, the *only* separating axis is the one parallel to the center-to-center line segment. As the spheres move apart, we can rotate the separating axis more and more in either direction. This is shown in Figure 13.11.

13.3.5.4 AABB versus AABB

To determine whether two AABBs are intersecting, we can again apply the separating axis theorem. The fact that the faces of both AABBs are guaranteed to lie parallel to a common set of coordinate axes tells us that if a separating axis exists, it will be one of these three coordinate axes.

So, to test for intersections between two AABBs, which we'll call A and B, we merely inspect the minimum and maximum coordinates of the two boxes along each axis independently. Along the x -axis, we have the two intervals

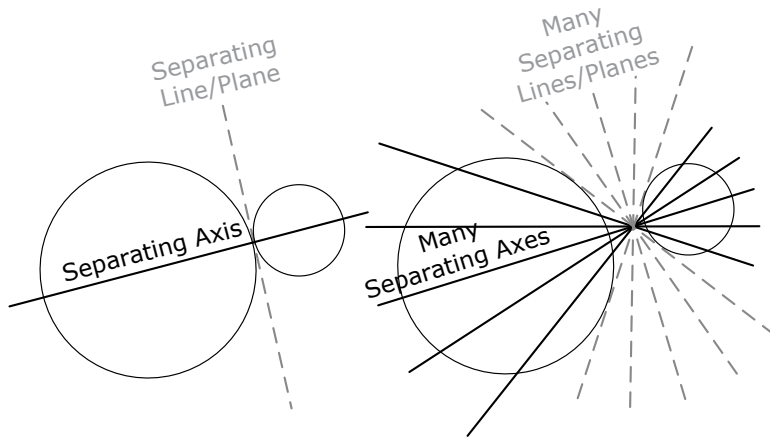


Figure 13.11. When two spheres are an infinitesimal distance apart, the only separating axis lies parallel to the line segment formed by the two spheres' center points.

$[x_{\min}^A, x_{\max}^A]$ and $[x_{\min}^B, x_{\max}^B]$, and we have corresponding intervals for the y - and z -axes. If the intervals overlap along *all three axes*, then the two AABBs are intersecting—in all other cases, they are not. Examples of intersecting and non-intersecting AABBs are shown in Figure 13.12 (simplified to two dimensions for the purposes of illustration). For an in-depth discussion of AABB collision, see http://www.gamasutra.com/features/20000203/lander_01.htm.

13.3.5.5 Detecting Convex Collisions: The GJK Algorithm

A very efficient algorithm exists for detecting intersections between arbitrary convex *polytopes* (i.e., convex polygons in two dimensions, or convex polyhedra in three dimensions). It is known as the GJK algorithm, named after its inventors, E. G. Gilbert, D. W. Johnson and S. S. Keerthi of the University of Michigan. Many papers have been written on the algorithm and its vari-

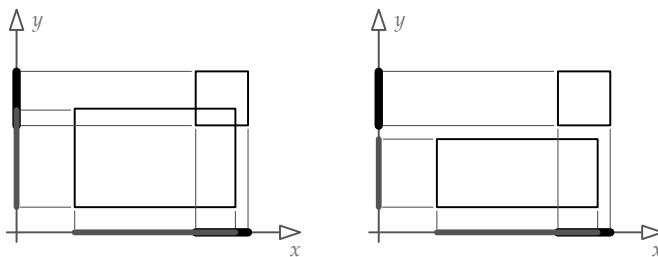


Figure 13.12. A two-dimensional example of intersecting and non-intersecting AABBs. Notice that even though the second pair of AABBs are intersecting along the x -axis, they are not intersecting along the y -axis.

ants, including the original paper (http://ieeexplore.ieee.org/xpl/freeabs_all.jsp?arnumber=2083), an excellent SIGGRAPH PowerPoint presentation by Christer Ericson (http://realtimecollisiondetection.net/pubs/SIGGRAPH04_Ericson_the_GJK_algorithm.ppt) and another great PowerPoint presentation by Gino van den Bergen (www.laas.fr/~nic/MOVIE/Workshop/Slides/Gino.vander.Bergen.ppt). However, the easiest-to-understand (and most entertaining) description of the algorithm is probably Casey Muratori's instructional video entitled "Implementing GJK," available online at <http://mollyrocket.com/849>. Because these descriptions are so good, I'll just give you a feel for the essence of the algorithm here and then direct you to the Molly Rocket website and the other references cited above for additional details.

The GJK algorithm relies on a geometric operation known as the *Minkowski difference*. This fancy-sounding operation is really quite simple: We take every point that lies within shape B and subtract it pairwise from every point inside shape A. The resulting set of points $\{(\mathbf{A}_i - \mathbf{B}_j)\}$ is the Minkowski difference.

The useful thing about the Minkowski difference is that, when applied to two convex shapes, it will *contain the origin* if and only if those two shapes intersect. Proof of this statement is a bit beyond our scope, but we can intuit why it is true by remembering that when we say two shapes A and B intersect, we really mean that there are points within A that are *also* within B. During the process of subtracting every point in B from every point in A, we would expect to eventually hit one of those shared points that lies within both shapes. A point minus itself is all zeros, so the Minkowski difference will contain the origin if (and only if) sphere A and sphere B have points in common. This is illustrated in Figure 13.13.

The Minkowski difference of two convex shapes is itself a convex shape. All we care about is the *convex hull* of the Minkowski difference, not all of the interior points. The basic procedure of GJK is to try to find a *tetrahedron* (i.e., a four-sided shape made out of triangles) that lies on the convex hull of the Minkowski difference and that encloses the origin. If one can be found, then the shapes intersect; if one cannot be found, then they don't.

A tetrahedron is just one case of a geometrical object known as a *simplex*. But don't let that name scare you—a simplex is just a collection of points. A single-point simplex is a point, a two-point simplex is a line segment, a three-point simplex is a triangle and a four-point simplex is a tetrahedron (see Figure 13.14).

GJK is an iterative algorithm that starts with a one-point simplex lying anywhere within the Minkowski difference hull. It then attempts to build higher-order simplexes that might potentially contain the origin. During each iteration of the loop, we take a look at the simplex we currently have and determine

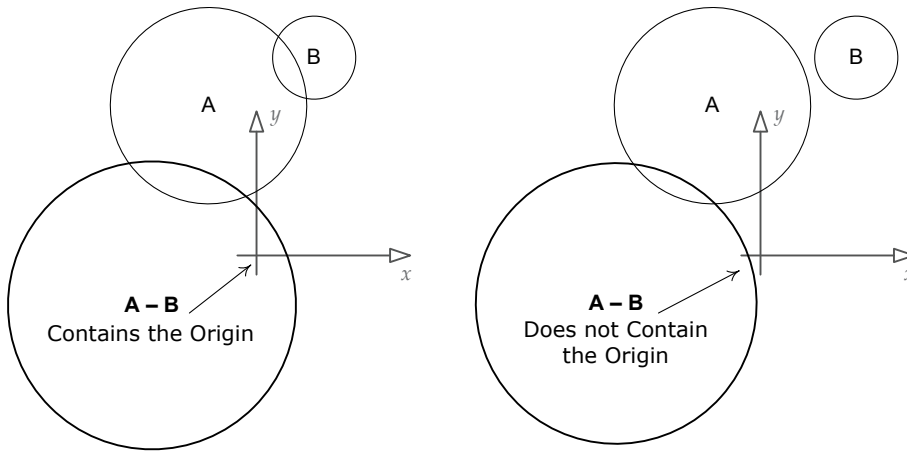


Figure 13.13. The Minkowski difference of two intersecting convex shapes contains the origin, but the Minkowski difference of two non-intersecting shapes does not.

in which direction the origin lies relative to it. We then find a *supporting vertex* of the Minkowski difference in that direction—i.e., the vertex of the convex hull that is closest to the origin in the direction we’re currently going. We add that new point to the simplex, creating a higher-order simplex (i.e., a point becomes a line segment, a line segment becomes a triangle and a triangle becomes a tetrahedron). If the addition of this new point causes the simplex to surround the origin, then we’re done—we know the two shapes intersect. On the other hand, if we are unable to find a supporting vertex that is closer to the origin than the current simplex, then we know that we can never get there, which implies that the two shapes do *not* intersect. This idea is illustrated in Figure 13.15.

To truly understand the GJK algorithm, you’ll need to check out the papers and video I referenced previously. But hopefully this description will whet your appetite for deeper investigation. Or, at the very least, you can impress

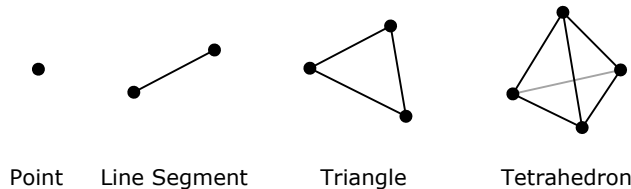


Figure 13.14. Simplexes containing one, two, three and four points.

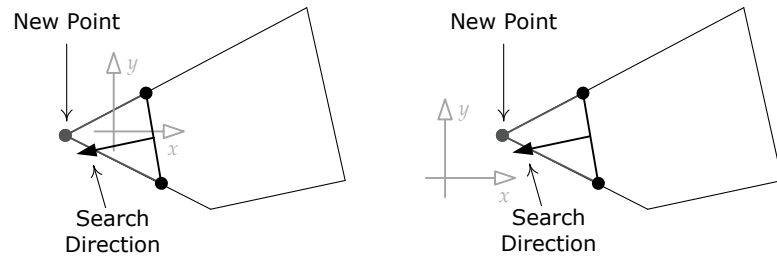


Figure 13.15. In the GJK algorithm, if adding a point to the current simplex creates a shape that contains the origin, we know the shapes intersect; if there is no supporting vertex that will bring the simplex any closer to the origin, then the shapes do not intersect.

your friends by dropping the name “GJK” at parties. (Just don’t try this at job interviews unless you really do understand the algorithm!)

13.3.5.6 Other Shape-Shape Combinations

We won’t cover any of the other shape-shape intersection combinations here, as they are covered well in other texts such as [14], [48] and [11]. The key point to recognize here, however, is that the *number* of shape-shape combinations is very large. In fact, for N shape types, the number of pairwise tests required is $O(N^2)$. Much of the complexity of a collision engine arises because of the sheer number of intersection cases it must handle. This is one reason why the authors of collision engines usually try to limit the number of primitive types—doing so drastically reduces the number of cases the collision detector must handle. (This is also why GJK is popular—it handles collision detection between *all* convex shape types in one fell swoop. The only thing that differs from shape type to shape type is the *support function* used in the algorithm.)

There’s also the practical matter of how to implement the code that selects the appropriate collision-testing function given two arbitrary shapes that are to be tested. Many collision engines use a *double dispatch* method (http://en.wikipedia.org/wiki/Double_dispatch). In single dispatch (i.e., virtual functions), the type of a single object is used to determine which concrete implementation of a particular abstract function should be called at runtime. Double dispatch extends the virtual function concept to two object types. It can be implemented via a two-dimensional function look-up table keyed by the types of the two objects being tested. It can also be implemented by arranging for a virtual function based on the type of object A to call a second virtual function based on the type of object B.

Let’s take a look at a real-world example. Havok uses objects known as *collision agents* (classes derived from `hkpCollisionAgent`) to handle specific

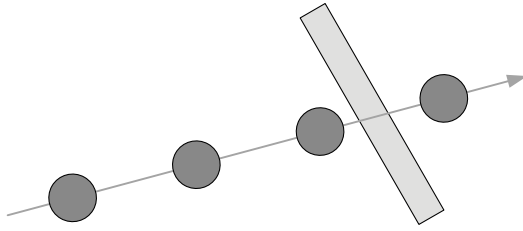


Figure 13.16. A small, fast-moving object can leave gaps in its motion path between consecutive snapshots of the collision world, meaning that collisions might be missed entirely.

intersection test cases. Concrete agent classes include `hkpSphereSphereAgent`, `hkpSphereCapsuleAgent`, `hkpGskConvexConvexAgent` and so on. The agent types are referenced by what amounts to a two-dimensional dispatch table, managed by the class `hkpCollisionDispatcher`. As you'd expect, the dispatcher's job is to efficiently look up the appropriate agent given a pair of collidables that are to be collision-tested and then call it, passing the two collidables as arguments.

13.3.5.7 Detecting Collisions between Moving Bodies

Thus far, we've considered only static intersection tests between stationary objects. When objects move, this introduces some additional complexity. Motion in games is usually simulated in discrete time steps. So one simple approach is to treat the positions and orientations of each rigid body as stationary at each time step and use static intersection tests on each "snapshot" of the collision world. This technique works as long as objects aren't moving too fast relative to their sizes. In fact, it works so well that many collision/physics engines, including Havok, use this approach by default.

However, this technique breaks down for small, fast-moving objects. Imagine an object that is moving so fast that it covers a distance *larger than its own size* (measured in the direction of travel) between time steps. If we were to overlay two consecutive snapshots of the collision world, we'd notice that there is now a gap between the fast-moving object's images in the two snapshots. If another object happens to lie within this gap, we'll miss the collision with it entirely. This problem, illustrated in Figure 13.16, is known as the "bullet through paper" problem, also known as "tunneling." The following sections describe a number of common ways to overcome this problem.

Swept Shapes

One way to avoid tunneling is to make use of *swept shapes*. A swept shape

is a new shape formed by the motion of a shape from one point to another over time. For example, a swept sphere is a capsule, and a swept triangle is a triangular prism (see Figure 13.17).

Rather than testing static snapshots of the collision world for intersections, we can test the swept shapes formed by moving the shapes from their positions and orientations in the previous snapshot to their positions and orientations in the current snapshot. This approach amounts to *linearly interpolating* the motion of the collidables between snapshots, because we generally sweep the shapes along line segments from snapshot to snapshot.

Of course, linear interpolation may not be a good approximation of the motion of a fast-moving collidable. If the collidable is following a curved path, then theoretically we should sweep its shape along that curved path. Unfortunately, a convex shape that has been swept along a curve is not itself convex, so this can make our collision tests much more complex and computationally intensive.

In addition, if the convex shape we are sweeping is rotating, the resulting swept shape is not necessarily convex, even when it is swept along a line segment. As Figure 13.18 shows, we *can* always form a convex shape by linearly extrapolating the extreme features of the shapes from the previous and current snapshots—but the resulting convex shape is not necessarily an accurate representation of what the shape really would have done over the time step. Put another way, a linear interpolation is not appropriate in general for rotating shapes. So unless our shapes are not permitted to rotate, intersection testing of swept shapes becomes much more complex and computationally intensive than its static snapshot-based counterpart.

Swept shapes can be a useful technique for ensuring that collisions are not missed between static snapshots of the collision world state. However, the results are generally inaccurate when linearly interpolating curved paths or

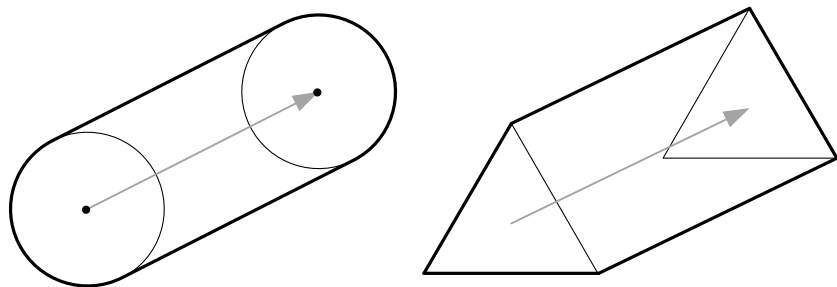


Figure 13.17. A swept sphere is a capsule; a swept triangle is a triangular prism.

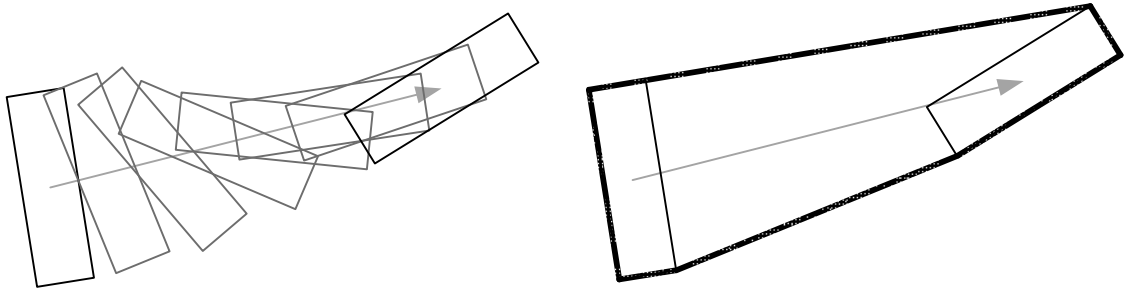


Figure 13.18. A rotating object swept along a line segment does not necessarily generate a convex shape (left). A linear interpolation of the motion does form a convex shape (right), but it can be a fairly inaccurate approximation of what actually happened during the time step.

rotating collidables, so more-detailed techniques may be required depending on the needs of the game.

Continuous Collision Detection (CCD)

Another way to deal with the tunneling problem is to employ a technique known as *continuous collision detection* (CCD). The goal of CCD is to find the earliest *time of impact* (TOI) between two moving objects over a given time interval.

CCD algorithms are generally iterative in nature. For each collidable, we maintain both its position and orientation at the previous time step and its position and orientation at the current time. This information can be used to linearly interpolate the position and rotation independently, yielding an approximation of the collidable's transform at any time between the previous and current time steps. The algorithm then searches for the earliest TOI along the motion path. A number of search algorithms are commonly used, including Brian Mirtich's *conservative advancement* method, performing a ray cast on the Minkowski sum, or considering the minimum TOI of individual feature pairs. Erwin Coumans of Sony Interactive Entertainment describes some of these algorithms in <http://gamedevs.org/uploads/continuous-collision-detection-and-physics.pdf> along with his own novel variation on the conservative advancement approach.

13.3.6 Performance Optimizations

Collision detection is a CPU-intensive task for two reasons:

1. The calculations required to determine whether two shapes intersect are themselves nontrivial.

2. Most game worlds contain a large number of objects, and the number of intersection tests required grows rapidly as the number of objects increases.

To detect intersections between n objects, the brute-force technique would be to test every possible pair of objects, yielding an $O(n^2)$ algorithm. However, much more efficient algorithms are used in practice. Collision engines typically employ some form of spatial hashing (<http://bit.ly/1fLtX1D>), spatial subdivision or hierarchical bounding volumes in order to reduce the number of intersection tests that must be performed.

13.3.6.1 Temporal Coherency

One common optimization technique is to take advantage of *temporal coherency*, also known as *frame-to-frame coherency*. When collidables are moving at reasonable speeds, their positions and orientations are usually quite similar from time step to time step. We can often avoid recalculating certain kinds of information every frame by caching the results across multiple time steps. For example, in Havok, collision agents (`hkpCollisionAgent`) are usually persistent between frames, allowing them to reuse calculations from previous time steps as long as the motion of the collidables in question hasn't invalidated those calculations.

13.3.6.2 Spatial Partitioning

The basic idea of spatial partitioning is to greatly reduce the number of collidables that need to be checked for intersection by dividing space into a number of smaller regions. If we can determine (in an inexpensive manner) that a pair of collidables do not occupy the same region, then we needn't perform more-detailed intersection tests on them.

Various hierarchical partitioning schemes, such as octrees, binary space partitioning trees (BSPs), kd-trees or sphere trees, can be used to subdivide space for the purposes of collision detection optimization. These trees subdivide space in different ways, but they all do so in a hierarchical fashion, starting with a gross subdivision at the root of the tree and further subdividing each region until sufficiently fine-grained regions have been obtained. The tree can then be walked in order to find and test groups of potentially colliding objects for actual intersections. Because the tree partitions space, we know that when we traverse down one branch of the tree, the objects in that branch cannot be colliding with objects in other sibling branches.

13.3.6.3 Broad Phase, Midphase and Narrow Phase

Havok uses a three-tiered approach to prune the set of collidables that need to be tested for collisions during each time step.

- First, gross AABB tests are used to determine which collidables are potentially intersecting. This is known as *broad phase* collision detection.
- Second, the coarse bounding volumes of compound shapes are tested. This is known as *midphase* collision detection. For example, in a compound shape composed of three spheres, the bounding volume might be a fourth, larger sphere that encloses the other spheres. A compound shape may contain other compound shapes, so in general a compound collidable has a bounding volume hierarchy. The midphase traverses this hierarchy in search of subshapes that are potentially intersecting.
- Finally, the collidables' individual primitives are tested for intersection. This is known as *narrow phase* collision detection.

The Sweep and Prune Algorithm

In all of the major collision/physics engines (e.g., Havok, ODE, PhysX), broad phase collision detection employs an algorithm known as *sweep and prune* (http://en.wikipedia.org/wiki/Sweep_and_prune). The basic idea is to sort the minimum and maximum dimensions of the collidables' AABBs along the three principal axes, and then check for overlapping AABBs by traversing the sorted lists. Sweep and prune algorithms can make use of frame-to-frame coherency (see Section 13.3.6.1) to reduce an $O(n \log n)$ sort operation to an expected $O(n)$ running time. Frame coherency can also aid in the updating of AABBs when objects rotate.

13.3.7 Collision Queries

Another responsibility of the collision detection system is to answer hypothetical questions about the collision volumes in the game world. Examples include the following:

- If a bullet travels from the player's weapon in a given direction, what is the first target it will hit, if any?
- Can a vehicle move from point A to point B without striking anything along the way?
- Find all enemy objects within a given radius of a character.

In general, such operations are known as *collision queries*.

The most common kind of query is a *collision cast*, sometimes just called a *cast*. (The terms *trace* and *probe* are other common synonyms for “cast.”) A cast determines what, if anything, a hypothetical object would hit if it were to be placed into the collision world and moved along a ray or line segment. Casts are different from regular collision detection operations because the entity being cast is not really in the collision world—it cannot affect the other objects in the world in any way. This is why we say that a collision cast answers *hypothetical* questions about the collidables in the world.

13.3.7.1 Ray Casting

The simplest type of collision cast is a *ray cast*, although this name is actually a bit of a misnomer. What we’re really casting is a *directed line segment*—in other words, our casts always have a start point ($\mathbf{p}_0$) and an end point ($\mathbf{p}_1$). The cast line segment is tested against the collidable objects in the collision world. If it intersects any of them, the contact point or points are returned.

Ray casting systems typically describe the line segment via its start point $\mathbf{p}_0$ and a delta vector $\mathbf{d}$ that, when added to $\mathbf{p}_0$, yields the end point $\mathbf{p}_1$. Any point on this line segment can be found via the following *parametric equation*, where the parameter t is permitted to vary between zero and one:

$$\mathbf{p}(t) = \mathbf{p}_0 + t\mathbf{d}, \quad t \in [0, 1].$$

Clearly, $\mathbf{p}_0 = \mathbf{p}(0)$ and $\mathbf{p}_1 = \mathbf{p}(1)$. In addition, any contact point along the segment can be uniquely described by specifying the value of the parameter t corresponding to the contact. Most ray casting APIs return their contact points as “ t values,” or they permit a contact point to be converted into its corresponding t by making an additional function call.

Most collision detection systems are capable of returning the *earliest contact*—i.e., the contact point that lies closest to $\mathbf{p}_0$ and corresponds to the smallest value of t . Some systems are also capable of returning a complete list of all collidables that were intersected by the ray or line segment. The information returned for each contact typically includes the t value, some kind of unique identifier for the collidable entity that was hit, and possibly other information such as the surface normal at the point of contact or other relevant properties of the shape or surface that was struck. One possible contact point data structure is shown below.

```
struct RayCastContact
{
    F32    m_t;                // the t value for this
                                // contact
```

```

    U32      m_collidableId; // which collidable did we
                          // hit?

    Vector m_normal;        // surface normal at
                          // contact pt.

    // other information...
};

```

Applications of Ray Casts

Ray casts are used heavily in games. For example, we might want to ask the collision system whether character A has a direct line of sight to character B. To determine this, we simply cast a directed line segment from the eyes of character A to the chest of character B. If the ray hits character B, we know that A can “see” B. But if the ray strikes some other object *before* reaching character B, we know that the line of sight is being blocked by that object. Ray casts are used by weapon systems (e.g., to determine bullet hits), player mechanics (e.g., to determine whether or not there is solid ground beneath the character’s feet), AI systems (e.g., line of sight checks, targeting, movement queries, etc.), vehicle systems (e.g., to locate and snap the vehicle’s tires to the terrain) and so on.

13.3.7.2 Shape Casting

Another common query involves asking the collision system how far an imaginary convex shape would be able to travel along a directed line segment before it hits something solid. This is known as a *sphere cast* when the volume being cast is a sphere, or a *shape cast* in general. (Havok calls them *linear casts*.) As with ray casts, a shape cast is usually described by specifying the start point $\mathbf{p}_0$, the distance to travel $\mathbf{d}$ and of course the type, dimensions and orientation of the shape we wish to cast.

There are two cases to consider when casting a convex shape.

1. The cast shape is already interpenetrating or contacting at least one other collidable, preventing it from moving away from its starting location.
2. The cast shape is not intersecting with anything else at its starting location, so it is free to move a nonzero distance along its path.

In the first scenario, the collision system typically reports the contact(s) between the cast shape and all of the collidables with which it is initially interpenetrating. These contacts might be *inside* the cast shape or on its *surface*, as shown in Figure 13.19.

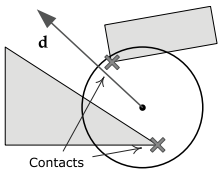


Figure 13.19. A cast sphere that starts in penetration will be unable to move, and possibly many contact points will lie inside the cast shape in general.

In the second case, the shape can move a nonzero distance along the line segment before striking something. Presuming that it hits something, it will usually hit only a single collidable. However, it is possible for a cast shape to strike more than one collidable simultaneously if its trajectory is just right. And of course, if the impacted collidable is a non-convex poly soup, the cast shape may end up touching more than one part of the poly soup simultaneously. We can safely say that no matter what kind of convex shape is cast, it is *possible* for the cast to generate multiple contact points. The contacts will always be on the *surface* of the cast shape in this case, never inside it (because we know that the cast shape was not interpenetrating anything when it started its journey). This case is illustrated in Figure 13.20.

As with ray casts, some shape casting APIs report only the *earliest* contact experienced by the cast shape, while others allow the shape to continue along its hypothetical path, returning all the contacts it experiences on its journey. This is illustrated in Figure 13.21.

The contact information returned by a shape cast is necessarily a bit more complex than it is for a ray cast. We cannot simply return one or more t values, because a t value only describes the location of the center point of the shape along its path. It tells us nothing of where, on the surface or interior of the shape, it came into contact with the impacted collidable. As a result, most shape casting APIs return both a t value and the actual contact point, along with other relevant information (such as which collidable was struck, the surface normal at the contact point, etc.).

Unlike ray casting APIs, a shape casting system must always be capable of reporting multiple contacts. This is because even if we only report the contact with the earliest t value, the shape may have touched multiple distinct collidables in the game world, or it may be touching a single non-convex collidable at more than one point. As a result, collision systems usually return an array

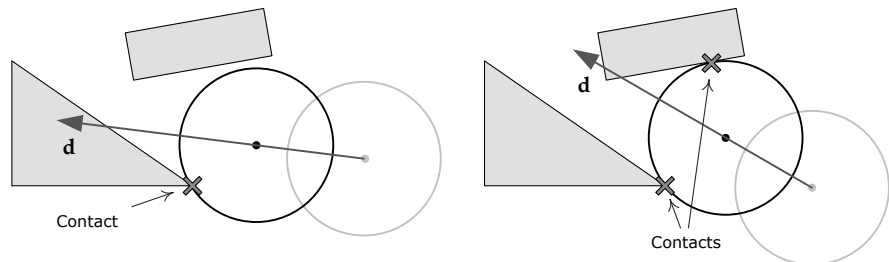


Figure 13.20. If the starting location of a cast shape is not interpenetrating anything, then the shape will move a nonzero distance along its line segment, and its contacts (if any) will always be on its surface.

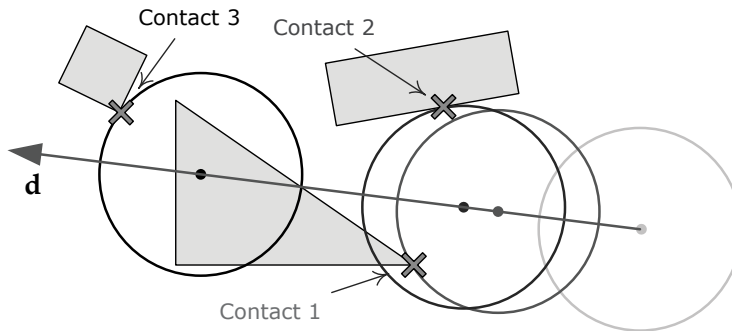


Figure 13.21. A shape casting API might return all contacts instead of only the earliest contact.

or list of contact point data structures, each of which might look something like this:

```
struct ShapeCastContact
{
    F32    m_t;           // the t value for this
                        // contact

    U32    m_collidableId; // which collidable did we
                        // hit?

    Point  m_contactPoint; // location of actual
                        // contact

    Vector m_normal;      // surface normal at
                        // contact pt.

    // other information...
};
```

Given a list of contact points, we often want to distinguish between the groups of contact points for each distinct t value. For example, the earliest contact is actually described by the group of contact points that all share the minimum t in the list. It's important to realize that collision systems may or may not return their contact points sorted by t . If it does not, it's almost always a good idea to sort the results by t manually. This ensures that if one looks at the first contact point in the list, it will be guaranteed to be among the earliest contact points along the shape's path.

Applications of Shape Casts

Shape casts are extremely useful in games. Sphere casts can be used to determine whether the virtual camera is in collision with objects in the game world. Sphere or capsule casts are also commonly used to implement character movement. For example, in order to slide the character forward on uneven terrain, we can cast a sphere or capsule that lies between the character's feet in the direction of motion. We can adjust it up or down via a second cast, to ensure that it remains in contact with the ground. If the sphere hits a very short vertical obstruction, such as a street curb, it can "pop up" over the curb. If the vertical obstruction is too tall, like a wall, the cast sphere can be slid horizontally along the wall. The final resting place of the cast sphere becomes the character's new location next frame.

13.3.7.3 Phantoms

Sometimes, games need to determine which collidable objects lie within some specific volume in the game world. For example, we might want the list of all enemies that are within a certain radius of the player character. Havok supports a special kind of collidable object known as a *phantom* for this purpose.

A phantom acts much like a shape cast whose distance vector $\mathbf{d}$ is zero. At any moment, we can ask the phantom for a list of its contacts with other collidables in the world. It returns this data in essentially the same format that would be returned by a zero-distance shape cast.

However, unlike a shape cast, a phantom is persistent in the collision world. This means that it can take full advantage of the temporal coherency optimizations used by the collision engine when detecting collisions between "real" collidables. In fact, the only difference between a phantom and a regular collidable is that it is "invisible" to all other collidables in the collision world (and it does not take part in the dynamics simulation). This allows it to answer hypothetical questions about what objects it would collide with were it a "real" collidable, but it is guaranteed not to have any effect on the other collidables—including other phantoms—in the collision world.

13.3.7.4 Other Types of Queries

Some collision engines support other kinds of queries in addition to casts. For example, Havok supports *closest point* queries, which are used to find the set of points on other collidables that are closest to a given collidable in the collision world.

13.3.8 Collision Filtering

It is quite common for game developers to want to enable or disable collisions between certain kinds of objects. For example, most objects are permitted to

pass through the surface of a body of water—we might employ a buoyancy simulation to make them float, or they might just sink to the bottom, but in either case we do not want the water’s surface to appear solid. Most collision engines allow contacts between collidables to be accepted or rejected based on game-specific criteria. This is known as *collision filtering*.

13.3.8.1 Collision Masking and Layers

One common filtering approach is to categorize the objects in the world and then use a look-up table to determine whether certain categories are permitted to collide with one another or not. For example, in Havok, a collidable can be a member of one (and only one) collision layer. The default collision filter in Havok, represented by an instance of the class `hkpGroupFilter`, maintains a 32-bit mask for each layer, each bit of which tells the system whether or not that particular layer can collide with one of the other layers.

13.3.8.2 Collision Callbacks

Another filtering technique is to arrange for the collision library to invoke a *callback function* whenever a collision is detected. The callback can inspect the specifics of the collision and make the decision to either allow or reject the collision based on suitable criteria. Havok also supports this kind of filtering. When contact points are first added to the world, the `contactPointAdded()` callback is invoked. If the contact point is later determined to be valid (it may not be if an earlier TOI contact was found), the `contactPointConfirmed()` callback is invoked. The application may reject contact points in these callbacks if desired.

13.3.8.3 Game-Specific Collision Materials

Game developers often need to categorize the collidable objects in the game world, in part to control how they collide (as with collision filtering) and in part to control other secondary effects, such as the sound that is made or the particle effect that is generated when one type of object hits another. For example, we might want to differentiate between wood, stone, metal, mud, water and human flesh.

To accomplish this, many games implement a collision shape categorization mechanism similar in many respects to the *material system* used in the rendering engine. In fact, some game teams use the term *collision material* to describe this categorization. The basic idea is to associate with each collidable surface a set of properties that defines how that particular surface should behave from a physical and collision standpoint. Collision properties can include sound and particle effects, physical properties like coefficient of restitution or